

A C++ Based Hand Gesture Recognition System For Computer Control (AYNIYH)

PROGRESS REPORT

BACHELOR OF TECHNOLOGY WITH RESEARCH

IN

COMPUTER SCIENCE AND ENGINEERING

SUBMITTED BY
Rittik Ganchaudhuri

Under the supervision of

Dr. Rajneesh Rani



Department of Computer Science and Engineering
Dr. B. R. Ambedkar National Institute of Technology Jalandhar
Punjab (India)

Introduction and Research Objective

The goal of this research is to solve the latency bottleneck inherent in prototype-stage gesture control systems. For Human-Computer Interaction (HCI) to feel instantaneous and immersive, research dictates that system latency must remain strictly minimized. Currently, the vast majority of academic HGR systems rely on Python due to its robust machine learning ecosystem. While excellent for training, Python's Global Interpreter Lock (GIL) and interpreted execution architecture introduce severe lag during live inference, rendering these systems unusable as professional mouse replacements. The AYNIIH project aims to prove that migrating the inference architecture to a compiled C++ environment while utilizing low-dimensional spatial landmarks resolves this latency gap.

Literature Review

To establish the methodology for this project, contemporary literature was systematically analyzed across three primary domains: HCI latency constraints, neural network architecture optimization, and production-environment deployment parameters.

1. HCI Usability and the Latency Bottleneck

The viability of Near-Field Gestures (NFG) as a replacement for traditional input devices is heavily dependent on system responsiveness. Harris et al. (2018) [1] utilized Fitts's Law to demonstrate that while gesture-based input is viable, it traditionally suffers from lower user productivity, reduced general comfort, and lower usability ratings compared to a physical mouse. These usability degradation factors are primarily driven by input lag and spatial tracking inaccuracies. To maintain the illusion of instantaneous control and preserve user immersion, HCI systems must overcome this latency threshold.

2. Architectural Efficiency: 1D CNNs vs. 2D Convolutions

Standard computer vision models frequently apply complex, multi-branch 2D Convolutional Neural Networks (CNNs) to raw pixel matrices, which is a computationally excessive approach for real-time tracking. Recent studies strongly advocate for extracting structured skeletal data prior to classification. Naya and Tanuwijaya (2025) [2] proved that when processing 63-dimensional static handlandmark vectors, simpler 1D CNNs significantly outperform complex models (like ResNet-1D and Inception-1D), achieving superior accuracy (97.61%) while providing the fastest inference speeds. This architectural efficiency is corroborated by Al Mudawi et al. (2024) [3], who successfully utilized 1D CNNs on hand skeletal coordinates to achieve highly robust, lightweight gesture recognition capable of continuous execution during daily-life routines.

3. Production Deployment and Resource Optimization

While Python dominates the training phase of machine learning, deploying models within its interpreted environment creates a severe performance bottleneck. Stacker et al. (2021) [4] directly addressed this by demonstrating that converting models to the standardized Open Neural Network Exchange (ONNX) format and deploying them via C++ hardware-accelerated runtimes significantly reduces inference latency compared

to native Python evaluation. Furthermore, optimizing the deployment environment is a critical constraint for background OS tasks. Alizadeh and Castor (2024) [5] established that transitioning from native Python frameworks to compiled execution engines drastically reduces energy consumption and CPU overhead. Consequently, utilizing a compiled C++ architecture is a mandatory engineering requirement for an always-on gesture mouse driver like AYNIIH.

4. Comparative Analysis

Table 1 highlights the architectural and methodological novelty of the proposed AYNIIH system compared to standard academic prototypes, directly addressing the limitations identified in the literature.

Table 1: Comparative Analysis against Standard HGR Prototypes

Feature / Metric	Standard Python Prototypes	AYNIIH(Proposed System)
Execution Environment	Python (Interpreted)	C++ (Compiled)
Model Format	PyTorch (.pt) / TensorFlow (.h5)	ONNX (.onnx)
Feature Extraction	Heavy 2D Image Convolutions	63D MediaPipe Hand Landmarks
Classification Model	ResNet or 2D CNN	1D Convolutional Neural Network
Inference Latency	High (Interpreter overhead)	Optimized (Direct memory access)

Current Progress and Methodology

The project has successfully completed the model training and initial C++ deployment phases:

1. **Data Extraction & Preprocessing:** Constructed a robust dataset (gestures.csv) mapping specific mouse-control gestures to 63-dimensional spatial coordinates (21 hand landmarks $\times$ 3 axes).
2. **Model Training:** Engineered and trained a lightweight 1D CNN (train cnn.py) in Python. The model evaluates spatial relationships between numerical landmarks rather than processing computationally expensive pixel matrices.
3. **C++ Porting & ONNX Integration:** Exported the trained model to the ONNX format. The core inference pipeline (main.cpp and hand tracker.cpp) was developed entirely in C++, utilizing ONNX Runtime to bypass Python's interpreter overhead and enable near-instantaneous gesture classification.

Identified Limitations

While the current architecture successfully mitigates inference latency, the following technical constraints require formal acknowledgment:

- 1. Lack of Temporal Context:** The 1D CNN classifies static frames based on instantaneous landmark coordinates. It lacks recurrent memory (such as an LSTM architecture) to understand fluid motion over time.
- 2. Scale and Rotational Variance:** Although wrist-relative coordinate normalization is implemented, extreme rotational angles or drastic shifts in camera depth may degrade the 1D CNN's classification accuracy.
- 3. Environmental Dependency:** The C++ pipeline remains fundamentally bottlenecked by the initial palm detection model; severe lighting conditions that cause MediaPipe to fail will result in a total loss of tracking data.

Next Steps and Future Work

To empirically validate the AYNIIYH system for the final research paper, the following quantitative experiments will be conducted in the subsequent phase:

- 1. Formal Latency Benchmarking:** Execute a millisecond-level inference comparison between the compiled C++ ONNX script and an equivalent Python-MediaPipe script to mathematically quantify the latency reduction.
- 2. Resource Overhead Profiling:** Measure and compare the background CPU and RAM consumption of the C++ executable versus a Python interpreter to prove viability as an efficient background OS task.
- 3. HCI Usability Testing (Fitts's Law):** Conduct a preliminary user study measuring target acquisition speed and error rates using the AYNIIYH gesture mouse versus a traditional optical mouse.

References

- [1] D. Harris, T. McIntyre, and K. Jones, “A Comparison of Near-Field Gesture, Touch, and Mouse Input,” in *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, 2018.
- [2] R. A. Naya and E. Tanuwijaya, “Comparative Analysis of 1D CNN Architectures for Guitar Chord Recognition from Static Hand Landmarks,” *Journal of Applied Informatics and Computing (JAIC)*, vol. 9, no. 6, pp. 3904–3918, Dec. 2025.
- [3] N. Al Mudawi *et al.*, “Innovative healthcare solutions: robust hand gesture recognition of daily life routines using 1D CNN,” *Frontiers in Bioengineering and Biotechnology*, vol. 12, Jul. 2024.
- [4] L. Stacker *et al.*, “Deployment of Deep Neural Networks for Object Detection on Edge AI Devices with Runtime Optimization,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) Workshops*, 2021, pp. 1015–1024.
- [5] N. Alizadeh and F. Castor, “Green AI: a Preliminary Empirical Study on Energy Consumption in DL Models Across Different Runtime Infrastructures,” in *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering (CAIN)*, Apr. 2024.